Tugas Kelompok: Implementasi WAW pada Simulator ILP

INF-1316 Organisasi dan Arsitektur Komputer

Anggota	NIM
Siti Nor Hamidah (RAW)	51240022
Naila Nur Azizah Musgamy (WAR)	51240034
Siti Nahdia Nahda (WAW)	51240025

Link GitHub tim: https://github.com/hamidahnursiti09-ux/ilp-hazard-simulator

1. Konsep WAW

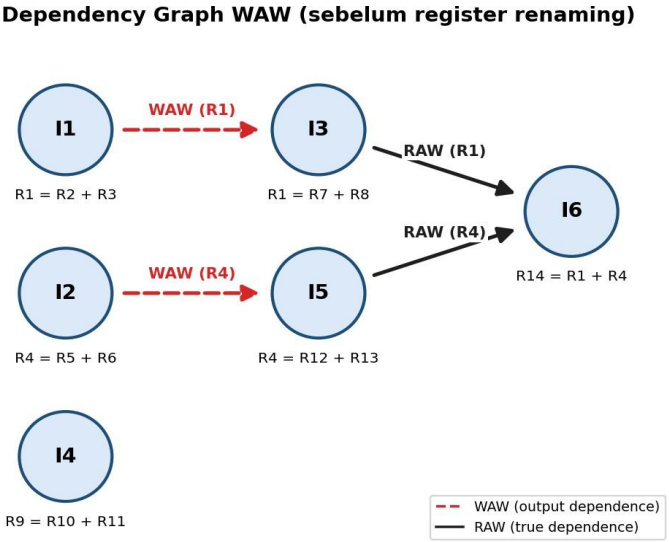
WAW (*Write After Write*) terjadi ketika dua instruksi menulis ke register yang sama. Instruksi yang lebih akhir tidak boleh selesai menulis lebih dulu daripada instruksi sebelumnya; jika urutannya tertukar, nilai akhir register menjadi salah. WAW disebut **output dependence** dan termasuk **name dependence**, sehingga dapat dihilangkan dengan **register renaming**.

2. Program Uji

Instruksi	Operasi
I1	R1 = R2 + R3
I2	R4 = R5 + R6
I3	R1 = R7 + R8
I4	R9 = R10 + R11
I5	R4 = R12 + R13
I6	R14 = R1 + R4

WAW: I1 dan I3 (R1), I2 dan I5 (R4). I6 membaca R1 dan R4 (RAW). Tidak ada WAR pada program ini.

3. Dependency Graph



Garis putus-putus merah = WAW, garis hitam = RAW.

4. Hasil Eksekusi Simulator

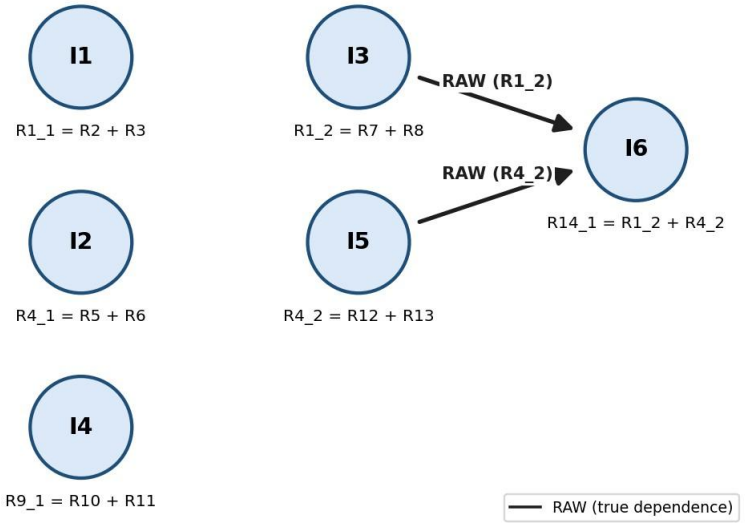
Skenario	Cycle	Speedup	Keterangan
Sequential	6	1.00x	Satu instruksi per cycle
ILP, WAW diabaikan	2 (tidak valid)	-	Salah: dua penulis register yang sama berjalan bersamaan
ILP + penanganan WAW (stall)	3	2.00x	Benar: I3 menunggu I1, I5 menunggu I2
ILP + register renaming	2	3.00x	Benar dan tercepat: WAW hilang

Jadwal ILP dengan stall WAW: Cycle 1: I1, I2, I4 | Cycle 2: I3, I5 | Cycle 3: I6.

5. Register Renaming

```
I1: R1_1 = R2 + R3
I2: R4_1 = R5 + R6
I3: R1_2 = R7 + R8
I4: R9_1 = R10 + R11
I5: R4_2 = R12 + R13
I6: R14_1 = R1_2 + R4_2
```

Dependency Graph setelah Register Renaming (WAW hilang)



Setelah renaming tidak ada lagi WAW; hanya RAW (true dependence) yang tersisa. Cycle 1: I1-I5 paralel, Cycle 2: I6.

6. Kesimpulan

(1) WAW terdeteksi bila dua instruksi memiliki register tujuan yang sama. (2) Jika simulator hanya memeriksa RAW, eksekusi paralel dapat salah. (3) Dengan penanganan WAW hasil benar (3 cycle, speedup 2x). (4) Dengan register renaming WAW hilang sehingga hanya 2 cycle (speedup 3x).